

## 2.3 Vim文本编辑器



[返回目录](#)

2.3.1

● 常用命令

2.3.2

● 编辑简单的文档

## 2.3 Vim文本编辑器

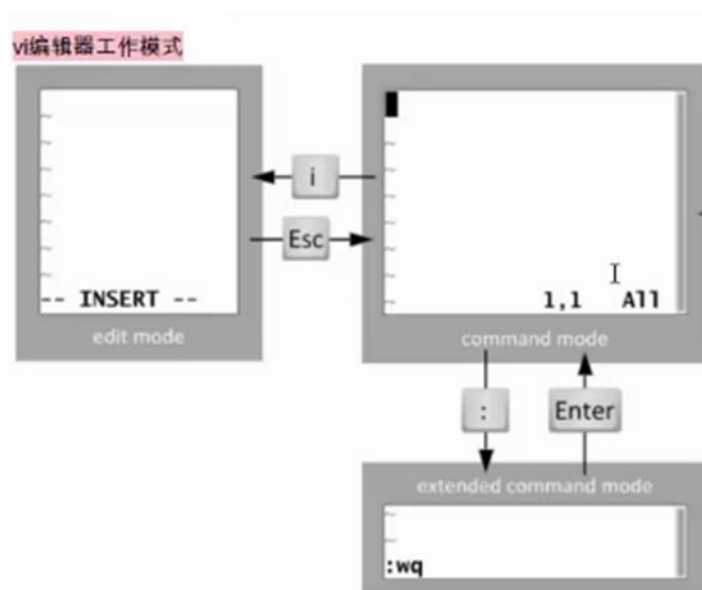
- Linux系统中的编辑器是对一些服务配置和文件编辑的工具，类似于Windows系统中的记事本。Vi编辑器是Linux上最基本的文本编辑器，工作在字符模式下，效率非常高。尽管在Linux上也有很多图形界面的编辑器可用，如gedit编辑器，但在系统和服务器管理中，Vi编辑器的功能是那些图形编辑器所无法比拟的。Vim是Vi的增强版，如果计算机上没有安装Vim，可以使用下面命令下载安装。

```
[root@qfedu ~]# which vim  
  
/usr/bin/vim  
  
[root@qfedu ~]# yum -y install vim-enhanced
```

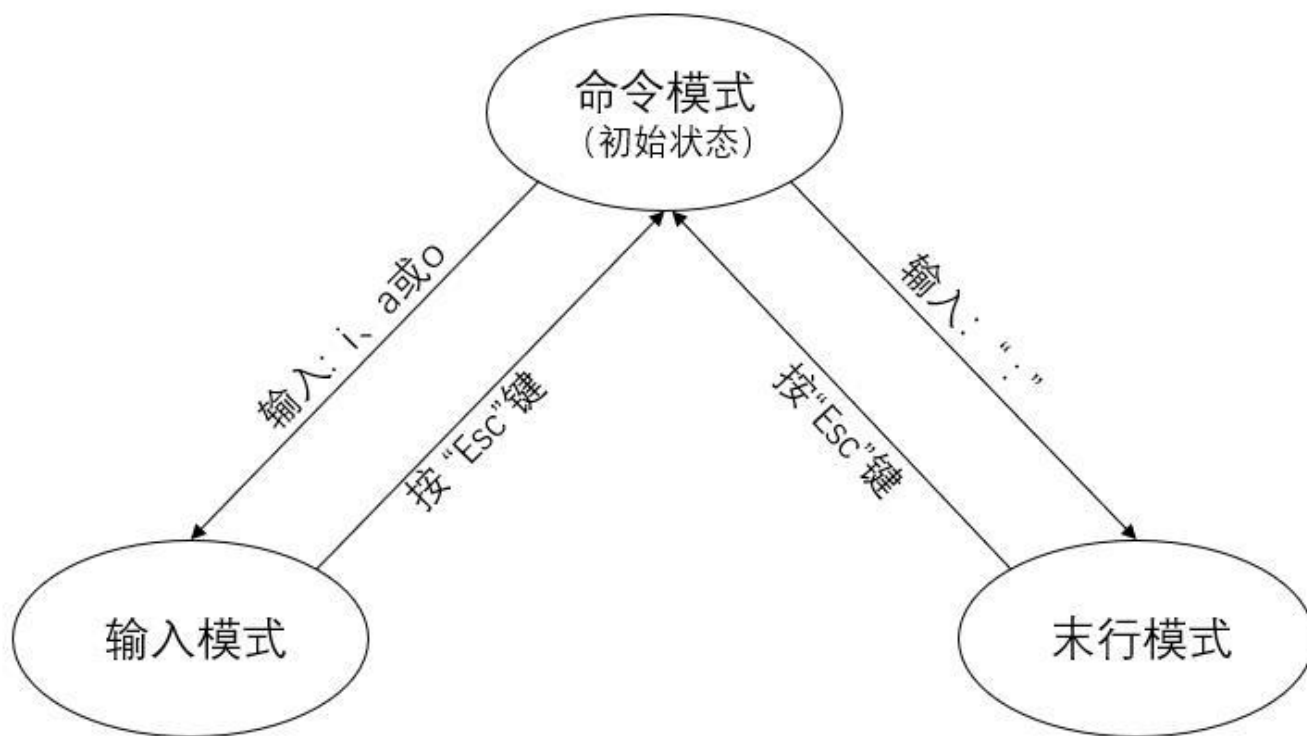
- Vim编辑器主要有三种模式：命令模式、编辑模式和末行模式（扩展命令模式）。  
命令模式：控制光标，对文件进行复制、粘贴、删除、查询等操作。
- 编辑模式：进行文本录入与更改。
- 末行模式：文档保存与退出，设置编辑环境。

## 2.3 Vim文本编辑器

- 每个模式下都有不同的命令快捷键，和图形相比有些抽象，但当用户熟练这些操作之后，工作效率就会比图形界面高很多。
- 当打开Vim编辑器后，默认进入命令模式，进入其他模式都是以命令模式发起的，例如，按i键进入编辑模式。此时如果要进入末行模式，用户需要先按Esc键返回命令模式，然后输入 “:” 即可，如图所示。



vim有3种基本工作模式：命令模式、输入模式和末行模式。用vim打开一个文件后，便处于命令模式。利用文本插入命令，如i、a、o等可以进入输入模式，按Esc键可以从输入模式退回命令模式。3种基本工作模式的转换如图所示。



# 三、项目实施

## 任务2-5 熟练使用vim编辑器

在命令行中执行命令“**vim filename**”，如果**filename**已存在，则**filename**被打开且显示其内容；如果**filename**不存在，则Vim在第一次存盘时自动在硬盘上新建**filename**文件。

Vim拥有5种编辑模式：**命令模式**、**输入模式**、**末行模式**、可视化模式、查询模式。

### 1. 命令模式（其它模式：**ESC**）

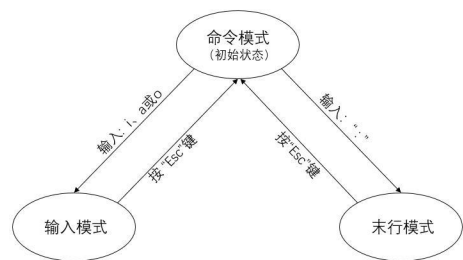
命令模式是用户进入Vim后的初始状态，在此模式中，可输入Vim命令，让Vim完成不同的工作。如：光标移动、删除字符和单词、段落复制等。也可对选定内容进行复制。从命令模式可切换到其它四种模式。也可从其它四种模式返回命令模式。在输入模式下按【**Esc**】键，或在末行模式输入了错误命令，都会回到命令模式，常用的操作及其说明见表2-7~表2-10。

### 2. 输入模式（命令模式：**a、i、o、A、I、O**）

在输入模式下，可对编辑的文件添加新的内容及修改，这是该模式的唯一功能，即文本输入。进入该模式，可按【**a/A**】、【**i/I**】或【**o/O**】键，它们的功能及其说明见表2-11。

### 3. 末行模式（在命令模式按冒号“**:**”）

主要用来进行一些文字编辑辅助功能，如字符串查找、替代和保存文件等，在命令模式中输入“**:**”字符，就可进入末行模式，在该模式下，若完成了输入的命令或命令出错，就会退出Vim或返回命令模式。常用的命令及其说明见表2-12。按【**Esc**】键返回命令模式。



# 三、项目实施

## 任务2-5 熟练使用vim编辑器

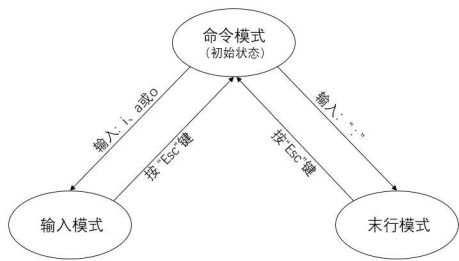
### 4. 可视化模式（命令模式：v）

在命令模式下键入v则进入可视化模式。在该模式下，移动光标以选定要操作的字符串，键入c剪切选定块的字符串，键入y复制选定块的字符串。

在命令模式中键入p，可将复制或剪切的内容粘贴在光标所在位置的右边。

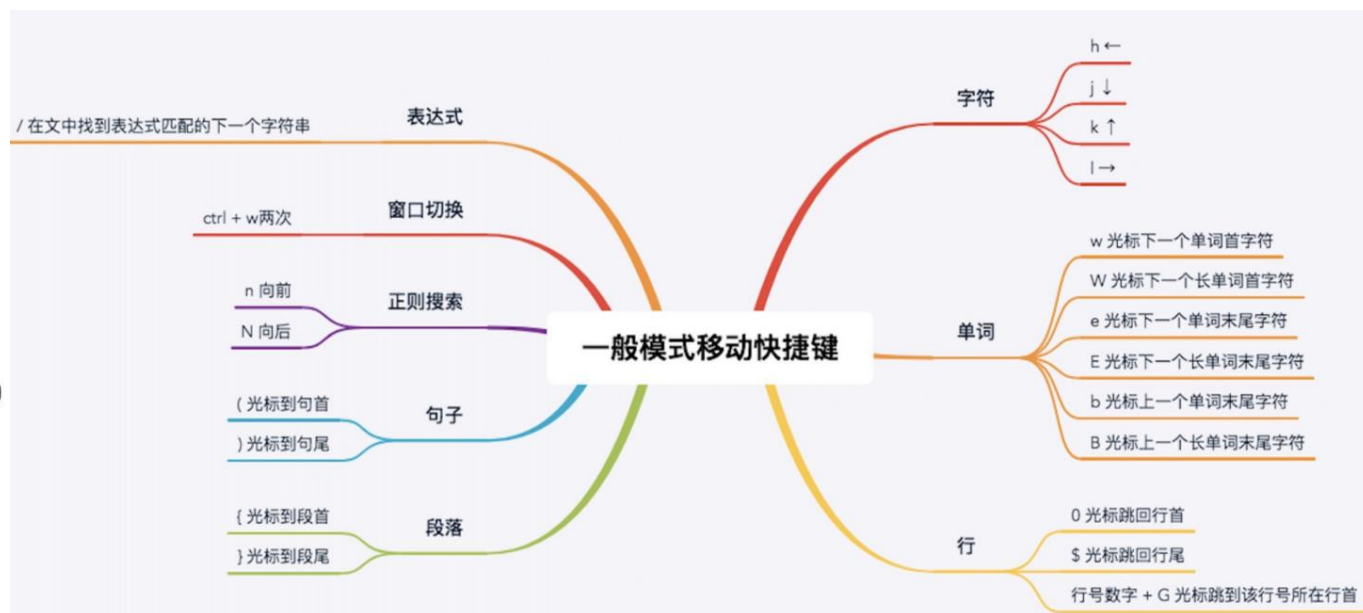
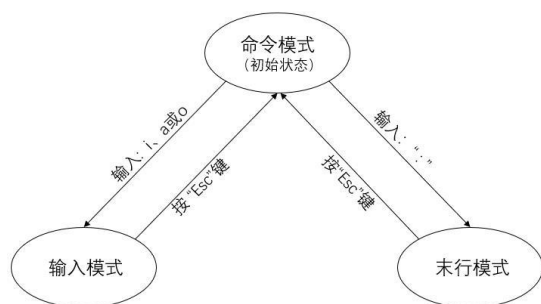
### 5. 查询模式（命令模式：?、/）

在命令模式中键入“/”、“?”等字符则进入查询模式（可以看成一种末行模式）。在该模式下，**向下/向上查询**文件中的某个关键字。在查找到后，可以用n/N**继续寻找**下一个/上一个关键字。常用的命令及其说明见表2-13。



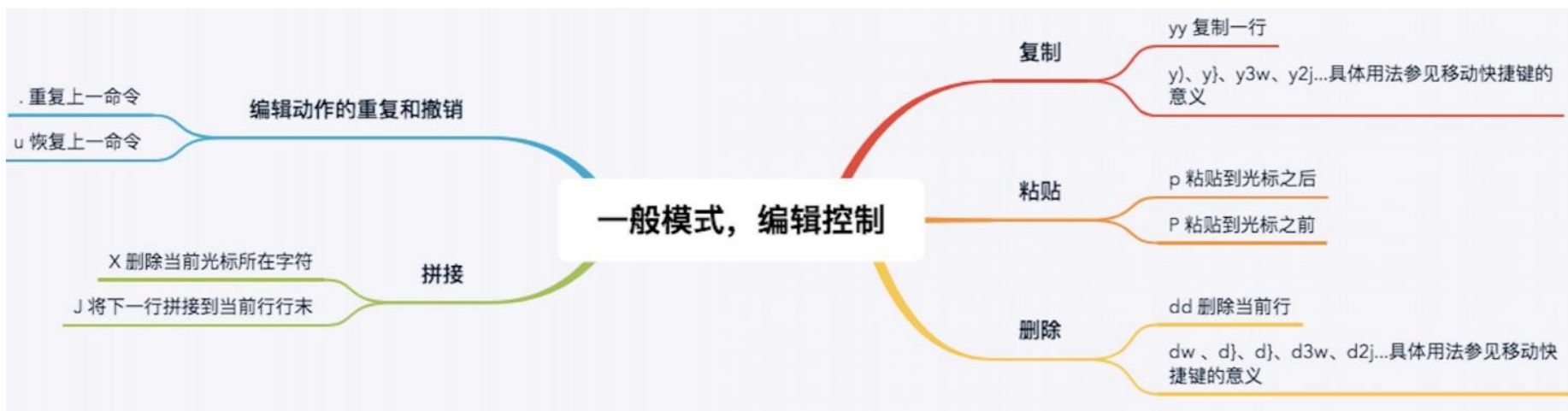
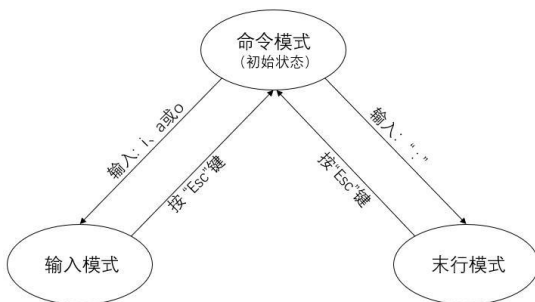
# 三、项目实施

## 任务2-5 熟练使用vim编辑器



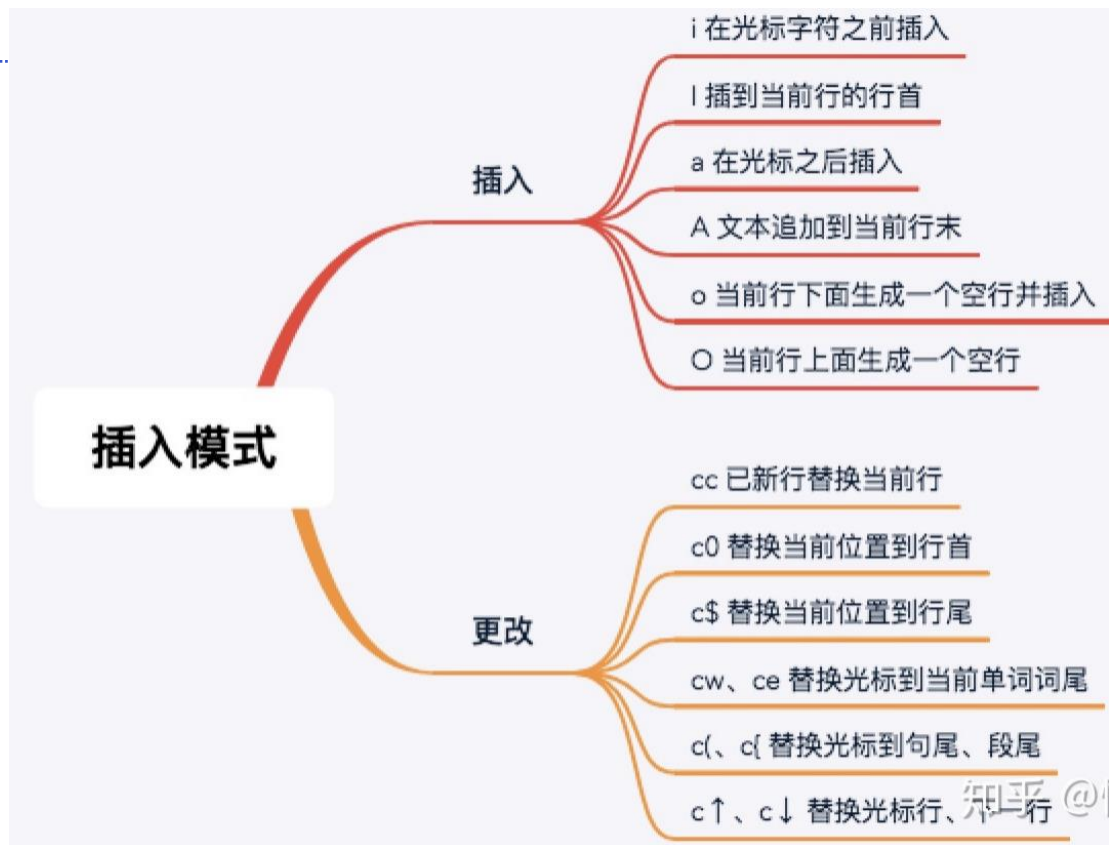
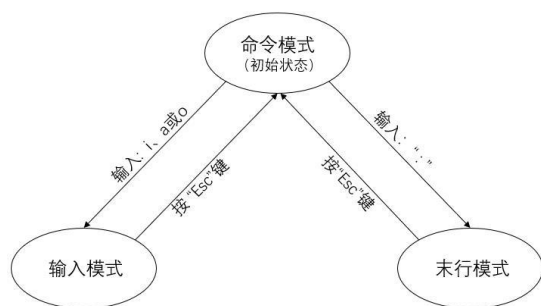
# 三、项目实施

## 任务2-5 熟练使用vim编辑器



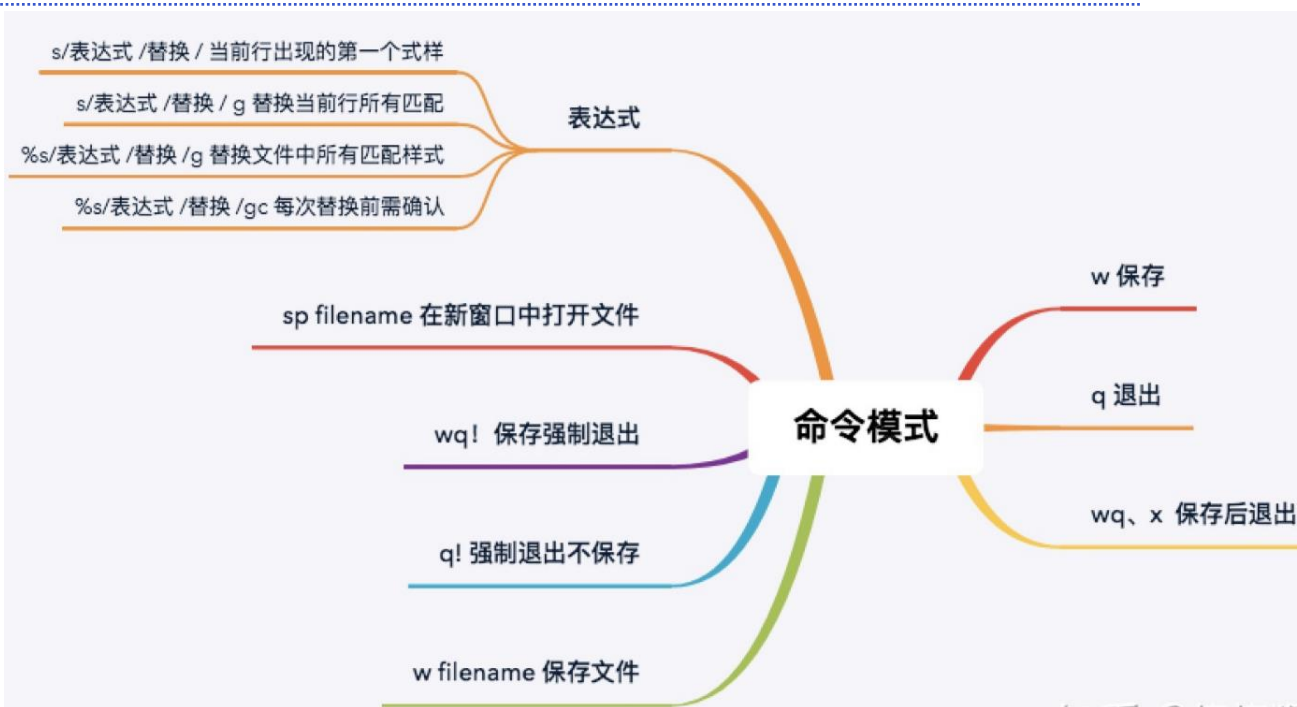
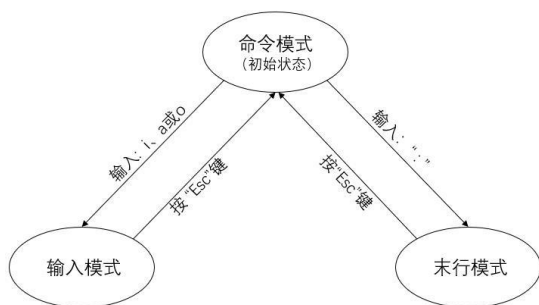
# 三、项目实施

## 任务2-5 熟练使用vim编辑器



# 三、项目实施

## 任务2-5 熟练使用vim编辑器



version 1.1  
April 1st, 06  
翻译: 2006-5-21

Esc

命令  
模式

## vi / vim 键盘图

|                         |               |                     |                 |                         |                          |              |                    |                   |             |                   |                          |                          |
|-------------------------|---------------|---------------------|-----------------|-------------------------|--------------------------|--------------|--------------------|-------------------|-------------|-------------------|--------------------------|--------------------------|
| ~ 转换<br>大小写             | ! 外部<br>过滤器   | @ 运行<br>宏           | # prev<br>ident | \$ 行尾                   | % 括号<br>匹配               | ^ "软"<br>行首  | & 重复<br>:s         | * next<br>ident   | ( 句首        | ) 下一<br>句首        | "soft" bol<br>down       | + 后一行<br>行首              |
| · 跳转到<br>标注             | 1             | 2                   | 3               | 4                       | 5                        | 6            | 7                  | 8                 | 9           | 0 "硬"<br>行首       | - 前一行<br>行首              | = 自动 <sup>3</sup><br>格式化 |
| Q 切换至<br>ex模式           | W 下一<br>单词    | E 词尾                | R 替换<br>模式      | T back<br>'till         | Y 拷贝<br>行                | U 撤消<br>行内命令 | I 到行首<br>插入        | O 分段<br>(前)       | P 粘贴<br>(前) | { 段首              | }                        | 段尾                       |
| q 录制<br>宏               | w 下一<br>单词    | e 词尾                | r 替换<br>字符      | t 'till                 | y 拷贝 <sup>1,3</sup><br>行 | u 撤消<br>命令   | i 插入<br>模式         | o 分段<br>(后)       | p 粘贴<br>(后) | [ 杂项 <sup>1</sup> | ]                        | 杂项                       |
| A 在行尾<br>附加             | S 删除行<br>并插入  | D 删除<br>至行尾         | F 行内字符<br>反向查找  | G 文尾/<br>行号             | H 屏幕<br>顶行               | J 合并<br>两行   | K 帮助               | L 屏幕<br>底行        | :           | ex 命令             | " 寄存器 <sup>1</sup><br>标识 | 行首/<br>列                 |
| a 附加                    | s 删除字符<br>并插入 | d 删除 <sup>1,3</sup> | f 行内字符<br>查找    | g 附加 <sup>6</sup><br>命令 | h ←                      | j ↓          | k ↑                | l →               | ;           | 重复<br>u/T/f/F     | ' 跳转到标<br>注的行首           | \ 未用!                    |
| Z 退出 <sup>4</sup>       | X 退格          | C 修改<br>至行末         | V 可视<br>行模式     | B 前一<br>单词              | N 查找<br>上一处              | M 屏幕<br>中间行  | < 反缩进 <sup>3</sup> | > 缩进 <sup>3</sup> | ?           | 向前<br>搜索          | .                        | 向后<br>搜索                 |
| Z 附加<br>命令 <sup>5</sup> | x 删除<br>(字符)  | c 修改 <sup>1,3</sup> | v 可视<br>模式      | b 前一<br>单词              | n 查找<br>下一处              | m 设置<br>标注   | , 反向<br>u/T/f/F    | .                 | 重复<br>命令    | /                 | .                        |                          |

动作 移动光标, 或者定义操作的范围

命令 直接执行的命令,  
红色命令 进入编辑模式

操作 后面跟随表示操作范围的指令

extra 特殊功能,  
需要额外的输入

q 后跟字符参数

w,e,b命令

小写(b): quux({foo, bar, baz});

大写(B): quux(foo, bar, baz);

主要ex命令:

:w (保存), :q (退出), :q! (不保存退出)

:ef (打开文件 f),

:%s/x/y/g ('y' 全局替换 'x'),

:h (帮助 in vim), :new (新建文件 in vim),

其它重要命令:

CTRL-R: 重复 (vim),

CTRL-F/-B: 上翻/下翻,

CTRL-E/-Y: 上滚/下滚,

CTRL-V: 块可视模式 (vim only)

可视模式:

漫游后对选中的区域执行操作 (vim only)

备注:

(1) 在 拷贝/粘贴/删除 命令前使用 "x (x=a..z,\*)  
使用命令的寄存器('剪贴板')

(如: "ay\$ 拷贝剩余的行内容至寄存器 'a')

(2) 命令前添加数字

多遍重复操作

(e.g.: 2p, d2w, 5i, d4j)

(3) 重复本字符在光标所在行执行操作

(dd = 删除本行, >> = 行首缩进)

(4) ZZ 保存退出, ZQ 不保存退出

(5) zt: 移动光标所在行至屏幕顶端,

zb: 底端, zz: 中间

(6) gg: 文首 (vim only),

gf: 打开光标处的文件名 (vim only)

原图: [www.viemu.com](http://www.viemu.com)

翻译: fdl (linuxsir)

## vim 快捷键

显示行号:

命令模式, 输入:set nu,回车

移动到第一行:

在命令模式, 连续敲击gg

移动到文档最后一行:

用shift按键+g

查找字符:

在命令模式, 输入/gzip

退出:

命令模式, 输入:wq

.....

# 三、项目实施

Vim替代方案--使用可视化~~gedit~~编辑器(vim过渡阶段使用，不建议长期使用)

**nohup gedit script2.sh &**

#请逐个执行如下命令，并截图

#用mkdir创建生成~/experiment/proj2/目录；

**mkdir** -p ~/experiment/proj2/

**ls** ~/experiment

#cd到上述目录；用touch在proj2目录下生成helloworld文件

**cd** ~/experiment/proj2/

**touch** helloworld

**ls**

#用echo在helloworld文件追加输入若干行文本信息并查看；

**echo** "abcdefg1234567890" > helloworld

**cat** helloworld

#用cp拷贝helloworld文件到helloworld-2；

**cp** helloworld helloworld-2 -rf

**ls**

#将helloworld-2通过mv命令更名为helloworld-3并查看

**mv** helloworld-2 helloworld-3

**cat** helloworld-3

File Edit View Search Terminal Help

[whxy@zhangsang ~]\$ cd /home/whxy/experiment/proj2

[whxy@zhangsang proj2]\$ ls

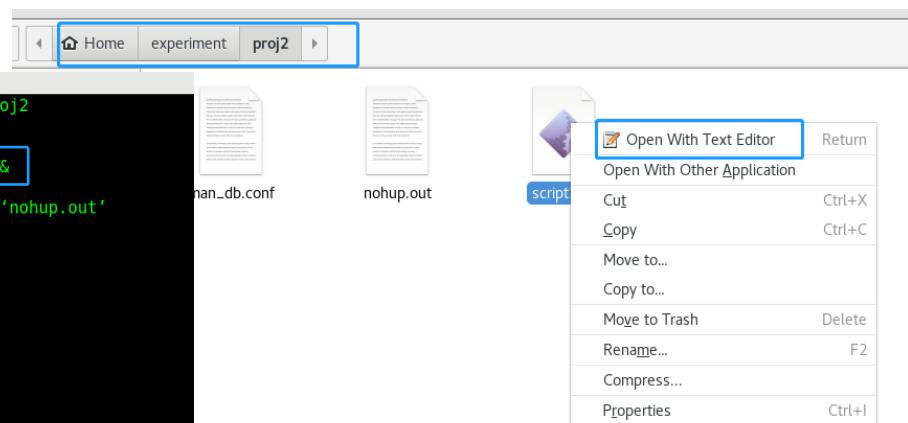
man\_db.conf nohup.out script2.sh

[whxy@zhangsang proj2]\$ **nohup gedit script2.sh &**

[1] 3854

nohup: ignoring input and appending output to 'nohup.out'

[whxy@zhangsang proj2]\$



## 2.3 Vim文本编辑器

### 2.3.1 常用命令

- 光标定位常用的命令如表所示。

| 命令      | 说明                  |
|---------|---------------------|
| h、j、k、l | 光标上下左右移动，也可用键盘自带方向键 |
| 0 和 \$  | 光标移动到行首、行尾          |
| gg 和 G  | 光标移动到第三行，移动到第 n 行   |
| /字符串    | (n N 可循环的)，快速定位到某一行 |
| /^d     | 定位首字母为 d 的行         |
| /txt\$  | 定位结尾为 txt 的行        |

## 2.3 Vim文本编辑器

### 2.3.1 常用命令

- 文本编辑常见的命令如表所示。

| 命令 | 说明         |
|----|------------|
| yy | 复制当前行      |
| dd | 删除当前行      |
| p  | 粘贴         |
| x  | 删除光标所在的字符  |
| D  | 从光标处删除到行尾  |
| u  | undo 撤销    |
| ^r | redo 重做    |
| r  | 可以用来修改一个字符 |

## 2.3 Vim文本编辑器

### 2.3.1 常用命令

- 命令和命令也可以组合，作用也是叠加的，例如，3yy、ygg、yG、dgg、dG等等，读者需平时多加练习。
- 从命令模式进入其他模式常见的命令如表所示。

| 命令 | 说明              |
|----|-----------------|
| o  | 进入编辑模式，光标下面另起一行 |
| a  | 进入编辑模式，光标后一位    |
| i  | 进入编辑模式，光标当前位置   |
| :  | 进入末行模式          |
| V  | 进入可视行模式         |
| v  | 进入可视模式          |
| ^v | 进入可视块模式         |
| R  | 进入替换模式          |

## 2.3 Vim文本编辑器

### 2.3.1 常用命令

- 末行模式常见的命令如表所示。

| 命令           | 说明                |
|--------------|-------------------|
| :w           | 保存                |
| :q           | 退出                |
| :wq          | 保存并退出             |
| :w!          | 强制保存              |
| :q!          | 不保存强制退出           |
| :wq!         | 强制保存退出            |
| :set nu      | 显示行号              |
| :set nonu    | 不显示行号             |
| :整数          | 跳到该行              |
| :s/abc/abd   | 该行第一个 abc 替换成 abd |
| :s/abc/abd/g | 该行所有 abc 替换成 abd  |
| :%字符串        | 在文本中从下至上搜索字符串     |
| :/字符串        | 在文本中从上至下搜索字符串     |

## 2.3 Vim文本编辑器

### 2.3.2 编辑简单的文档

- 用Vim编辑器编辑文档首先要给文档命名，此处命名为qfedu.txt。如果文档已经创建，则打开；如果文档不存在，此命令将创建一个文档，如图所示。



## 2.3 Vim文本编辑器

### 2.3.2 编辑简单的文档

- 打开文档后，默认进入命令模式，不能编辑文本，需要切换到编辑模式，按i键进入编辑模式，如图所示。



## 2.3 Vim文本编辑器

### 2.3.2 编辑简单的文档

- 进入编辑模式后，左下角出现“--插入--”，可随意输入文本内容，Vim编辑器不会把文本内容当作命令执行，如图所示。



## 2.3 Vim文本编辑器

### 2.3.2 编辑简单的文档

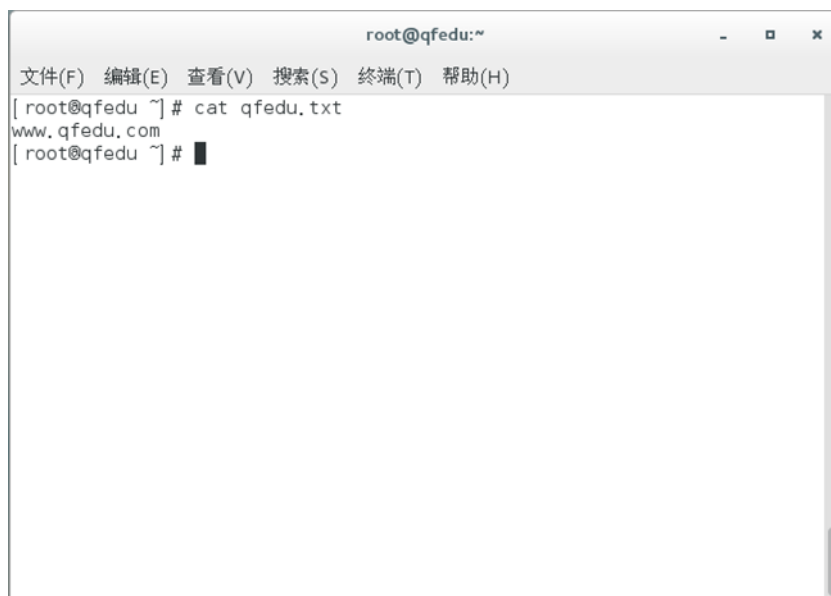
- 在编写完之后，保存退出。首先按Esc键切换到命令模式，然后再输入“:”，进入末行模式，最后输入“wq”，完成保存退出，如图所示。



## 2.3 Vim文本编辑器

### 2.3.2 编辑简单的文档

- 保存退出后，使用cat命令就可以查看刚才输入的文本，如图所示。

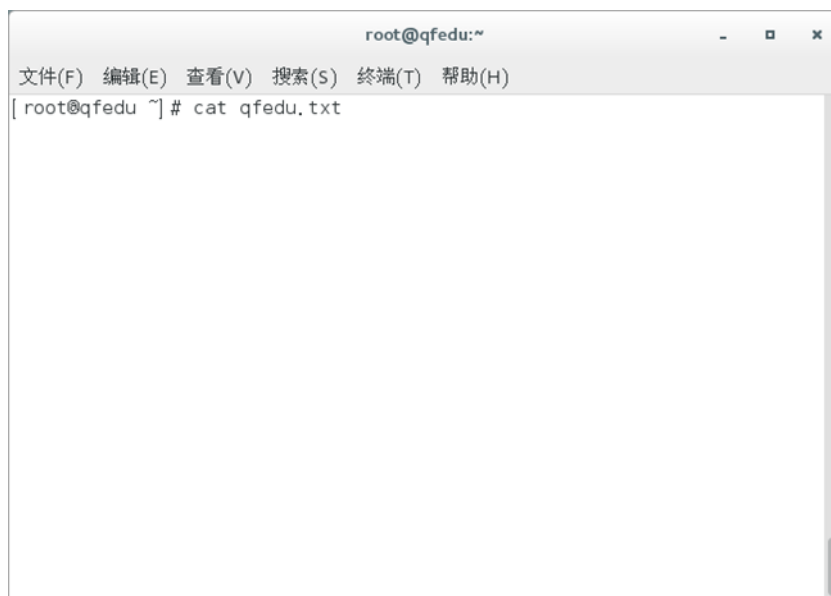


```
root@qfedu:~  
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)  
[ root@qfedu ~] # cat qfedu.txt  
www.qfedu.com  
[ root@qfedu ~] #
```

## 2.3 Vim文本编辑器

### 2.3.2 编辑简单的文档

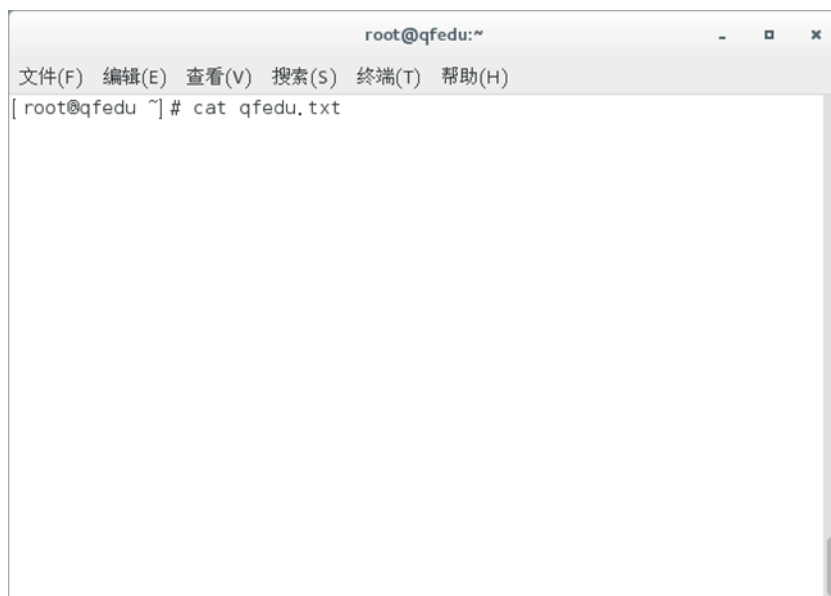
- 再次进入文档，可以继续编辑文件。按键盘方向键上快速翻出这条命令，如图所示。



## 2.3 Vim文本编辑器

### 2.3.2 编辑简单的文档

- 按o键进入编辑模式，可以另起一行并添加内容，如图所示。



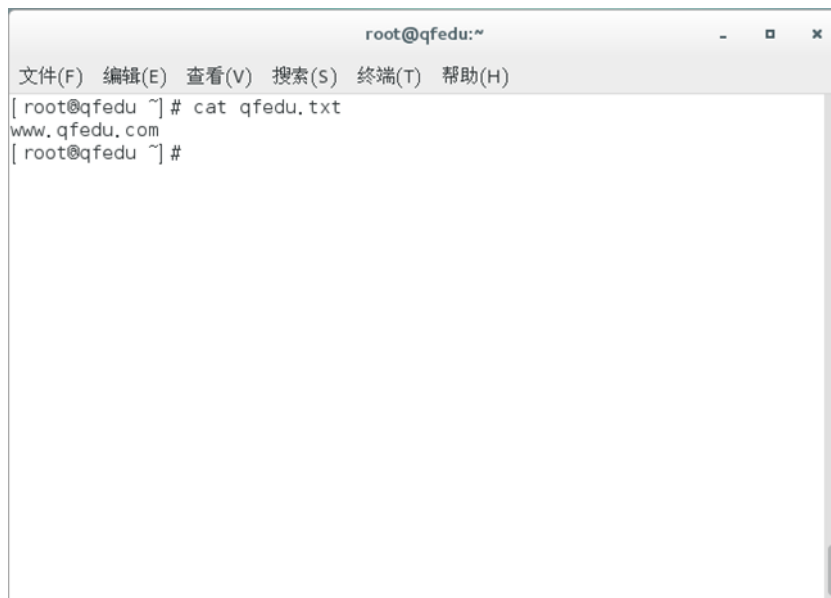




## 2.3 Vim文本编辑器

### 2.3.2 编辑简单的文档

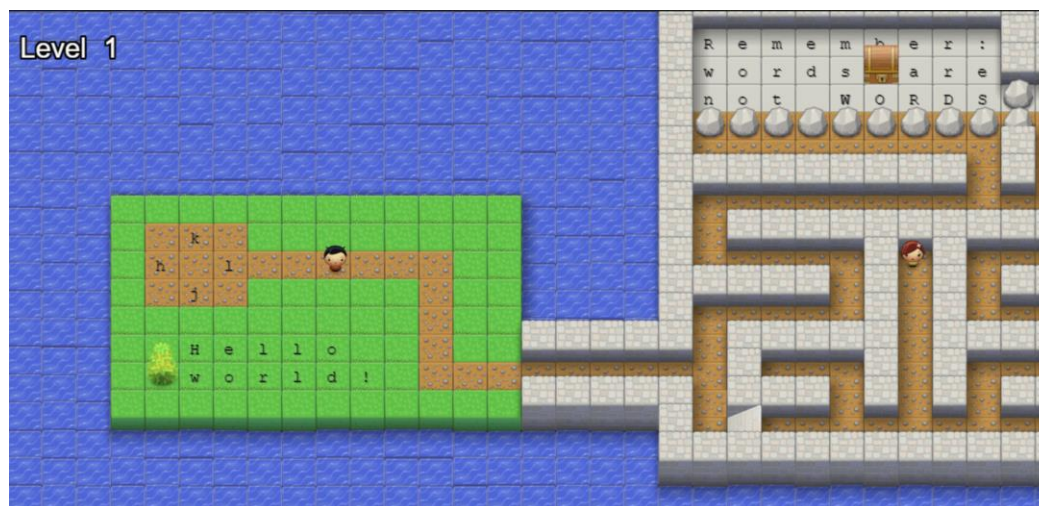
- 最后再次查看文本，发现并未保存第二次添加内容，只显示原来的内容，如图所示。



```
root@qfedu:~  
文件(F) 编辑(E) 查看(V) 搜索(S) 终端(T) 帮助(H)  
[ root@qfedu ~] # cat qfedu.txt  
www.qfedu.com  
[ root@qfedu ~] #
```

# Vim编辑器（P34-39）

- Vim附加练习：如果时间空余，请按照课本P36-39加强练习，或者完成下述趣味练习
- <https://vim-adventures.com/>



# Vim编辑器（P34-39）

---

- vim笔试题
- <https://www.jhq8.cn/s/vim/>
- 针对你感兴趣的若干习题，可以总结并提供心得和记录，截图在本区域；
- 期末考试vim的习题来源，部分来自这里

**下一堂课讲授**

## 2.4 文件时间

- 回到熟悉的Windows系统，在windows下新建一个文件，保存文件的同时也会保存文件的创建时间、修改时间、访问时间等，如图所示。



## 2.4 文件时间

- 在Linux系统下，一个文件也有三种时间：访问时间、修改时间、状态时间。stat命令可以查看文件的详细信息，具体如下所示。

```
[root@qfedu ~]# stat /etc/hostname
```

```
File: '/etc/hostname'
```

```
Size: 6          Blocks: 8          IO Block: 4096   regular file
```

```
Device: fd00h/64768d   Inode: 16931194   Links: 1
```

```
Access: (0644/-rw-r--r--)  Uid: (  0/   root)   Gid: (  0/   root)
```

```
Context: system_u:object_r:hostname_etc_t:s0
```

```
Access: 2018-04-02 09:29:54.065475006 +0800
```

```
Modify: 2018-03-30 14:17:48.875787110 +0800
```

```
Change: 2018-03-30 14:17:48.912788257 +0800
```

```
Birth: -
```

## 2.4 文件时间

- 从输出结果中可以看到系统保存了三个时间：Access time（访问时间）、Modify time（状态时间）、Change time（修改时间）。
- 在Linux系统中，文件是没有创建时间的，如果新创建一个文件，它的三个时间都是与创建的时间相同，具体如下所示。

```
[root@qfedu ~]# stat abc.txt

File: 'abc.txt'

Size: 4          Blocks: 8          IO Block: 4096   regular file

Device: fd00h/64768d    Inode: 35505558    Links: 1

Access: (0644/-rw-r--r--)  Uid: (   0/   root)   Gid: (   0/   root)

Context: unconfined_u:object_r:admin_home_t:s0

Access: 2018-04-08 18:39:49.771126373 +0800

Modify: 2018-04-08 18:39:49.771126373 +0800

Change: 2018-04-08 18:39:49.771126373 +0800

Birth: -
```

## 2.4 文件时间

- **修改时间**：文件的内容被最后一次修改的时间，“ls -l”命令显示出来的文件时间就是这个时间，当使用Vim对文件进行编辑之后保存，它的ctime就会相应地改变。
- **访问时间**：对文件进行一次读操作，它的访问时间就会改变。例如，cat、less等操作，但是state与ls命令对atime不会有影响。
- **状态时间**：当文件的状态被改变时，状态时间就会改变。使用chmod、chown等改变文件属性时，会改变文件的mtime。
- 以前的RHEL（Red Hat Enterprise Linux）版本，只要读取文件，就会刷新时间，这种时间的变化专业术语叫“磁盘的IO操作”，就是写磁盘，访问一次写一次。举例说明，千锋教育网站有上万个网页有上万个网页，访问一次时间就会刷新一次，最后会导致大量IO操作，这样的做的积极意义并不大，由此带来的消极意义却是明显的，大大增加了磁盘IO的工作量。

## 2.4 文件时间

- 从RHEL6开始，`atime`延迟修改，刷新时间必须满足下列两个条件之一：自上次`atime`修改后已过去86400秒；发生写操作。这个改变作用是很大的，例如，`find`命令查看根目录下5天以内被改过的文件（选取部门查询内容显示），具体如下所示。

```
[root@qfedu ~]# find / -mtime -5

/home/linux/.local/share/zeitgeist/activity.sqlite-shm
/home/linux/.local/share/zeitgeist/activity.sqlite-wal
...
/var/log/syslog
/var/log/auth.log
/var/log/kern.log
```

## 2.5 文件类型

- 前面提到过文件的颜色类别，但通过颜色判断文件的类型是不一定正确。Linux系统中文件是没有扩展名，修改无实际意义的扩展名无法修改文件的本质，具体如下所示。

```
[root@qfedu ~]# ls
anaconda-ks.cfg  initial-setup-ks.cfg
a.txt  b.txt
[root@qfedu ~]# file a.txt
a.txt: ASCII text //是txt文件
[root@qfedu ~]# file anaconda-ks.cfg
anaconda-ks.cfg: ASCII text //也是txt文件
[root@qfedu ~]# mv a.txt a.jpg //更改文件的扩展名为jpg格式
[root@qfedu ~]# ls
anaconda-ks.cfg  initial-setup-ks.cfg
a.jpg  b.txt
[root@qfedu ~]# file a.jpg
a.jpg: ASCII text //依然是txt文件
```

## 2.5 文件类型

- 使用 “ls -l” 命令查看文件名，看第一个字符，开头为 “-” 的普通文件（如文本文件、二进制文件、压缩文件、图片等），开头为 “d” 的是目录文件（蓝色），具体如下所示。

```
[root@qfedu ~]# ls -l /etc/
total 1348
-rw-r--r--. 1 root root    5090 Nov  5  2016 DIR_COLORS
-rw-r--r--. 1 root root    5725 Nov  5  2016 DIR_COLORS.256color
-rw-r--r--. 1 root root    4669 Nov  5  2016 DIR_COLORS.lightbgcolor
-rw-r--r--. 1 root root      94 Apr 29  2015 GREP_COLORS
-rw-r--r--. 1 root root    842 Nov  6  2016 GeoIP.conf
-rw-r--r--. 1 root root    858 Nov  6  2016 GeoIP.conf.default
drwxr-xr-x. 8 root root    145 Mar 30 12:46 NetworkManager
drwxr-xr-x. 2 root root     92 Mar 30 12:52 PackageKit
drwxr-xr-x. 2 root root     25 Mar 30 12:46 UPower
drwxr-xr-x. 6 root root    103 Mar 30 12:45 X11
drwxr-xr-x. 3 root root    101 Mar 30 12:45 abrt
.....省略部分文件.....
```

## 2.5 文件类型

- 开头为“b”的是设备文件（块设备），存储设备硬盘、U盘、/dev/sda、/dev/sda1；“c”表示设备文件（字符设备），打印机、终端、/dev/tty1、/dev/zero；“s”表示套接字文件；“p”表示管道文件；“l”表示链接文件（淡蓝色）。

```
[root@qfedu ~]# ll /dev/sda c
brw-rw----. 1 root disk 8, 0 Apr  2 09:29 /dev/sda

[root@qfedu ~]# ll /dev/zero
crw-rw-rw-. 1 root root 1, 5 Apr  2 09:29 /dev/zero

[root@qfedu ~]# ll /dev/log
srw-rw-rw-. 1 root root 0 Apr  2 09:29 /dev/log

[root@qfedu ~]# ll /run/dmeventd-client
prw----- 1 root root 0 Apr  2 09:29 /run/dmeventd-client

[root@qfedu ~]# ll /etc/grub2.cfg
lrwxrwxrwx. 1 root root 22 Mar 30 12:52 /etc/grub2.cfg
-> ../boot/grub2/grub.cfg
```

## 2.5 文件类型

- 使用file命令查看文件类型，如文本文件、二进制文件、管道文件、设备文件、链接文件等，具体如下所示。

```
[root@qfedu ~]# file /etc/hostname
```

```
/etc/hostname: ASCII text
```

```
[root@qfedu ~]# file /dev/sda
```

```
/dev/sda: block special
```

```
[root@qfedu ~]# file /dev/zero
```

```
/dev/zero: character special
```

## 2.5 文件类型

- 使用stat命令查看文件的详细属性，例如，文件的名称、大小、权限、atime、ctime、mtime等，具体如下所示。

```
[root@qfedu ~]# stat /etc/hostname

File: '/etc/hostname' //文件名字

Size: 6 //大小          Blocks: 8 //占块数    IO Block: 4096   regular file

Device: fd00h/64768d    Inode: 16931194   Links: 1

Access: (0644/-rw-r--r--) //权限  Uid: ( 0/ root) //所有者  Gid: ( 0/ root)

Context: system_u:object_r:hostname_etc_t:s0

Access: 2018-04-02 09:29:54.065475006 +0800

Modify: 2018-03-30 14:17:48.875787110 +0800

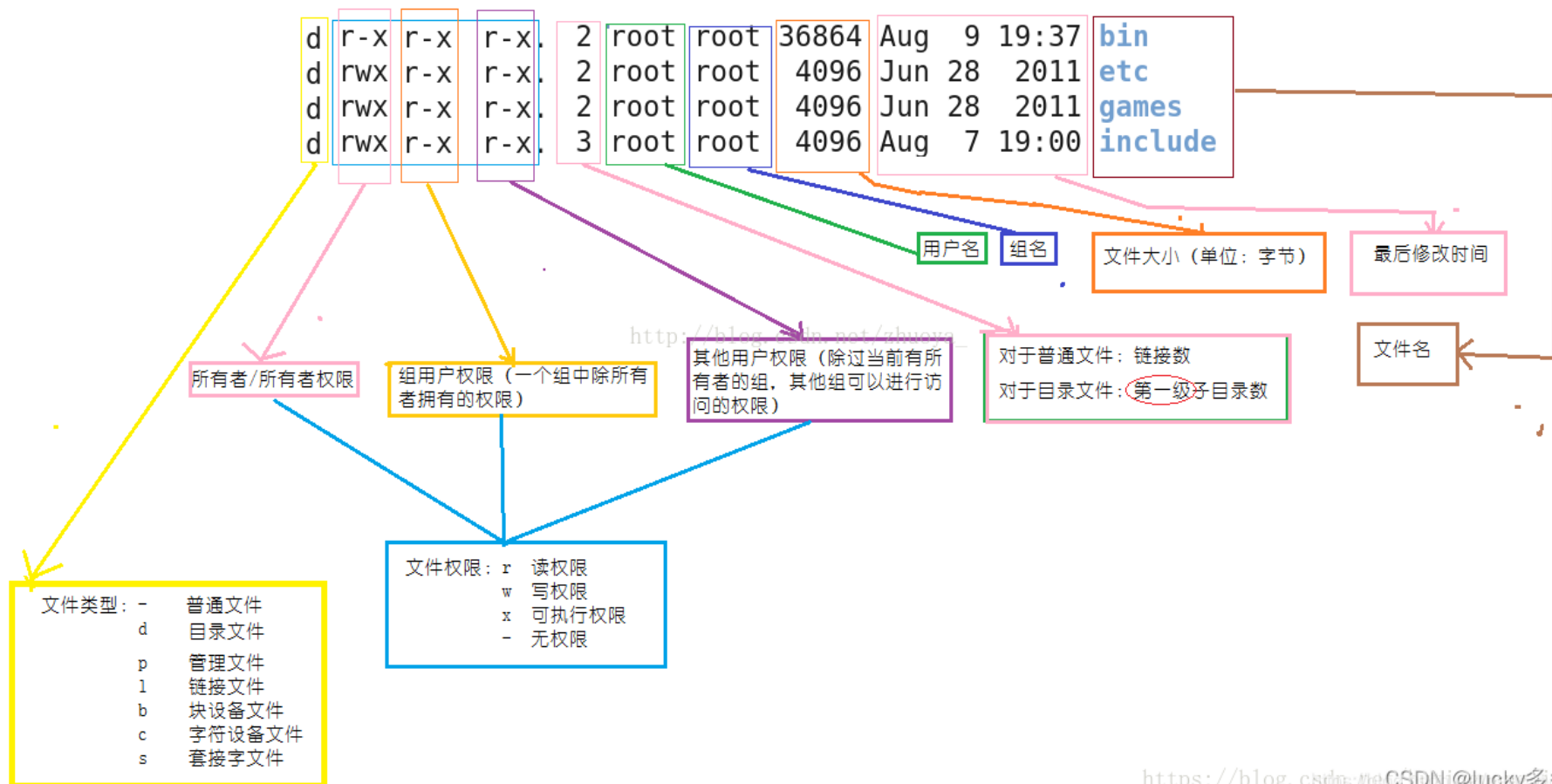
Change: 2018-03-30 14:17:48.912788257 +0800

Birth: -
```

# 常见文件类型

| 文件类型       | 描述                                  | 示例                                          |
|------------|-------------------------------------|---------------------------------------------|
| 普通文件       | 用来在辅助存储设备（如磁盘）上存储信息和数据              | 包含程序源代码（用C、C++、Java等语言所编写）、可执行程序、图片、声音、图像等  |
| 目录文件       | 用于表示和管理系统中的文件，目录文件中包含一些文件名和子目录名     | /root、/home                                 |
| 链接文件       | 用于不同目录下文件的共享                        | 当创建一个已存在文件的符号链接时，系统就创建一个链接文件，这个链接文件指向已存在的文件 |
| 设备文件       | 用来访问硬件设备                            | 包括键盘、硬盘、光驱、打印机等                             |
| 命名管道（FIFO） | 是一种特殊类型的文件，Linux系统下，进程之间通信可以通过该文件完成 |                                             |

在Linux中执行命令ls -l（以列表的形式显示文件）后，文件中各个信息代表的含义图解



# 高频考试题

**1.Linux中常用的文件类型有哪几种（列举至少5种）？如何区分？**

**回答:**

常见七种类型，

通过执行ls -l 文件名 //看第一个字符

格式            说明

- 普通文件(文本, 二进制, 压缩, 图片, 日志等)

d 目录文件

b 设备文件(块设备)存储设备硬盘 /dev/sda1, /dev/sda2

c 设备文件(字符设备)打印机, 终端 /dev/tty1, /dev/zero

s 套接字文件, 进程间通信(socket)

p 管道文件

l 链接文件